

SQL PROJECT – RETAIL STORE

SUBMITTED BY: LOVISH KOTHARI

MODULE: SQL MINI PROJECT

PROJECT: SQL PROJECT RETAIL STORE

PROJECT OVERVIEW

1. CREATE THE RETAIL STORE DATABASE.

2. INSERTED THE PROVIDED SAMPLE DATA.

3. SOLVED 42 SQL BUSINESS QUERIES.

4. INCLUDE OUTPUT SCREENSHOTS FOR EACH QUERY.

LEVEL – 1 (BASICS)

Q1. Retrieve customer names and emails for email marketing?

This helps the marketing team extract basic customer contact details for campaigns.

```
SELECT name, email

FROM customers;
```

OUTPUT(SCREENSHOT)

```
1     # LEVEL - 1 BASIC SELECT & SIMPLE FILTERS
2
3     # Q1. Retrieve customer names and emails for email marketing?
4     select name, email
5     from customers;
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com

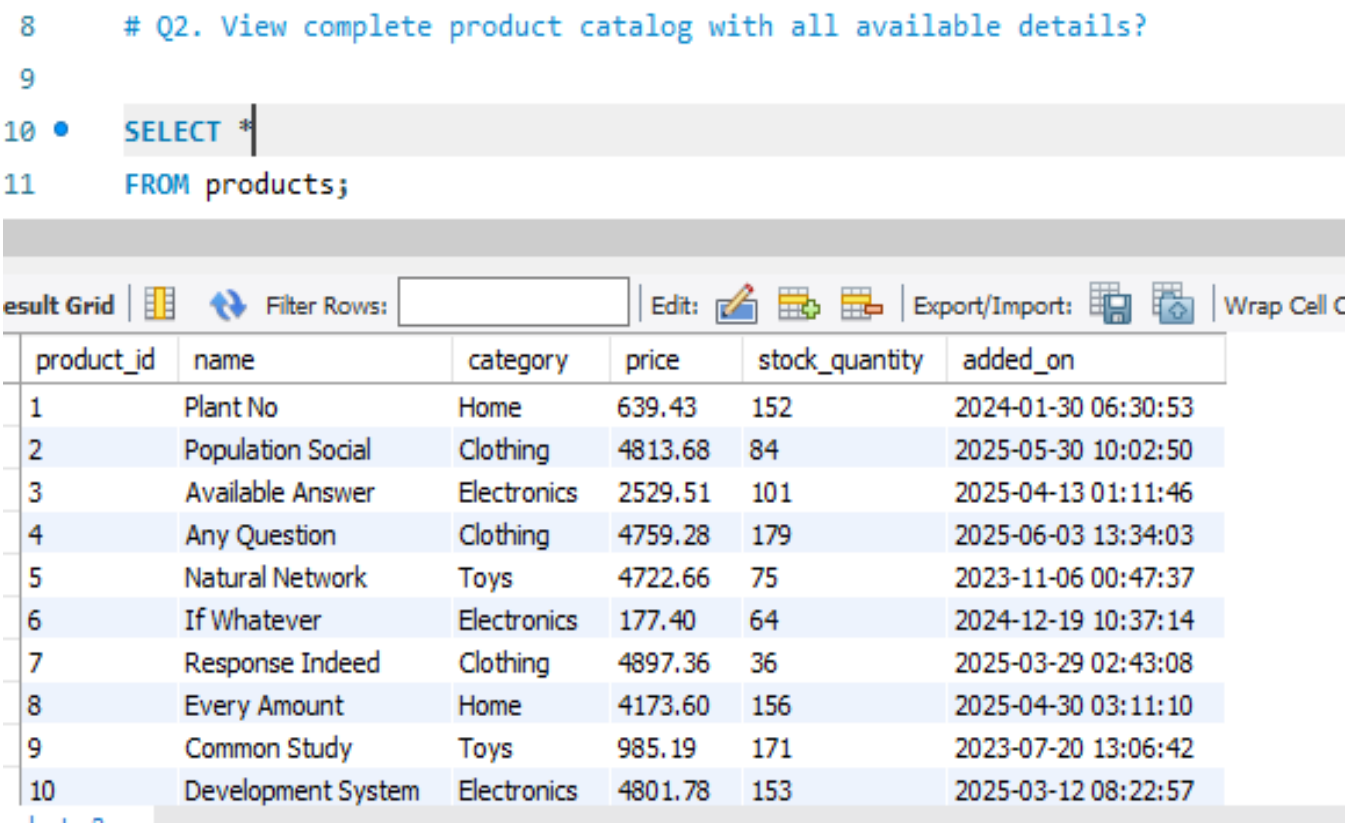
Q2. View complete product catalog with all available detail?

The product manager may want to review all product listings in one go.

```
SELECT *

FROM products;
```

OUTPUT(SCREENSHOT)



```
8     # Q2. View complete product catalog with all available details?
9
10    SELECT *
11    FROM products;
```

esult Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell C

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57

Q3. List all unique product categories?

Useful for analyzing the range of departments or for creating filters on the website

```
SELECT DISTINCT category

FROM products;
```

OUTPUT(SCREENSHOT)

```
15      # Q3. List all unique product categories?
16
17 •    SELECT DISTINCT category
18      FROM products;
19
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

category
Home
Clothing
Electronics
Toys
Books

Q4. Show all products priced above ₹1,000?

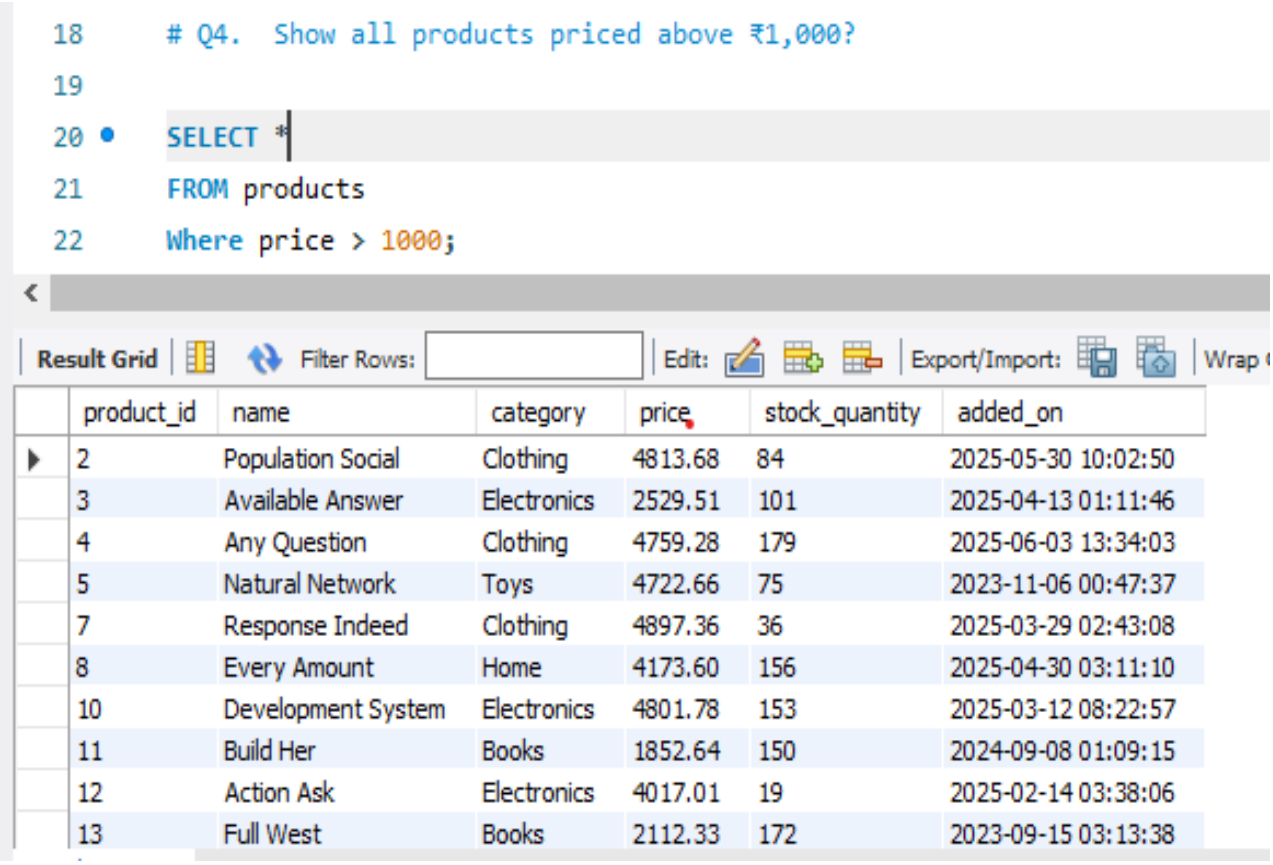
This helps identify high-value items for premium promotions or pricing strategy reviews.

```
SELECT *

FROM products

Where price > 1000;
```

OUTPUT(SCREENSHOT)



```
18    # Q4.  Show all products priced above ₹1,000?
19
20    SELECT *
21    FROM products
22    Where price > 1000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38

Q5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)?

A merchandising team might need this to create a mid-tier pricing campaign

```
SELECT *

FROM products

WHERE price between 2000
and 5000;
```

OUTPUT(SCREENSHOT)

```
24      # Q5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)?
25
26 •    SELECT *
27      FROM products
28      WHERE price between 2000 and 5000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
32	Factor Where	Clothing	2295.14	167	2023-07-07 00:59:39
33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
39	Data Add	Books	4063.38	62	2025-05-26 23:39:16
41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
42	Weight Enter	Clothing	3875.18	60	2024-01-26 08:21:55
45	Suffer We	Electronics	3657.43	113	2024-07-25 12:40:57
46	Age Treatment	Home	3728.83	160	2023-12-01 10:51:52
48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
49	Rest Something	Books	3077.82	181	2023-08-09 03:42:42

Q6. Fetch data for specific customer IDs (e.g., from loyalty program list)?

This is used when customer IDs are pre-selected from another system.

```
SELECT *

FROM customers

WHERE customer_id IN (2,5,8,24,29);
```

OUTPUT(SCREENSHOT)

```
30      # Q6. Fetch data for specific customer IDs (e.g., from loyalty program list)?
31
32 •    SELECT *
33      FROM customers
34      WHERE customer_id IN (2,5,8,24,29);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email	phone	created_at
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
24	Brandy Wright	user24@example.com	853.520.8915x61	2024-07-09 17:09:19
29	Kara Zavala	user29@example.com	434-985-5776x61	2023-09-27 15:57:09

Q7. Identify customers whose names start with the letter ‘A’?

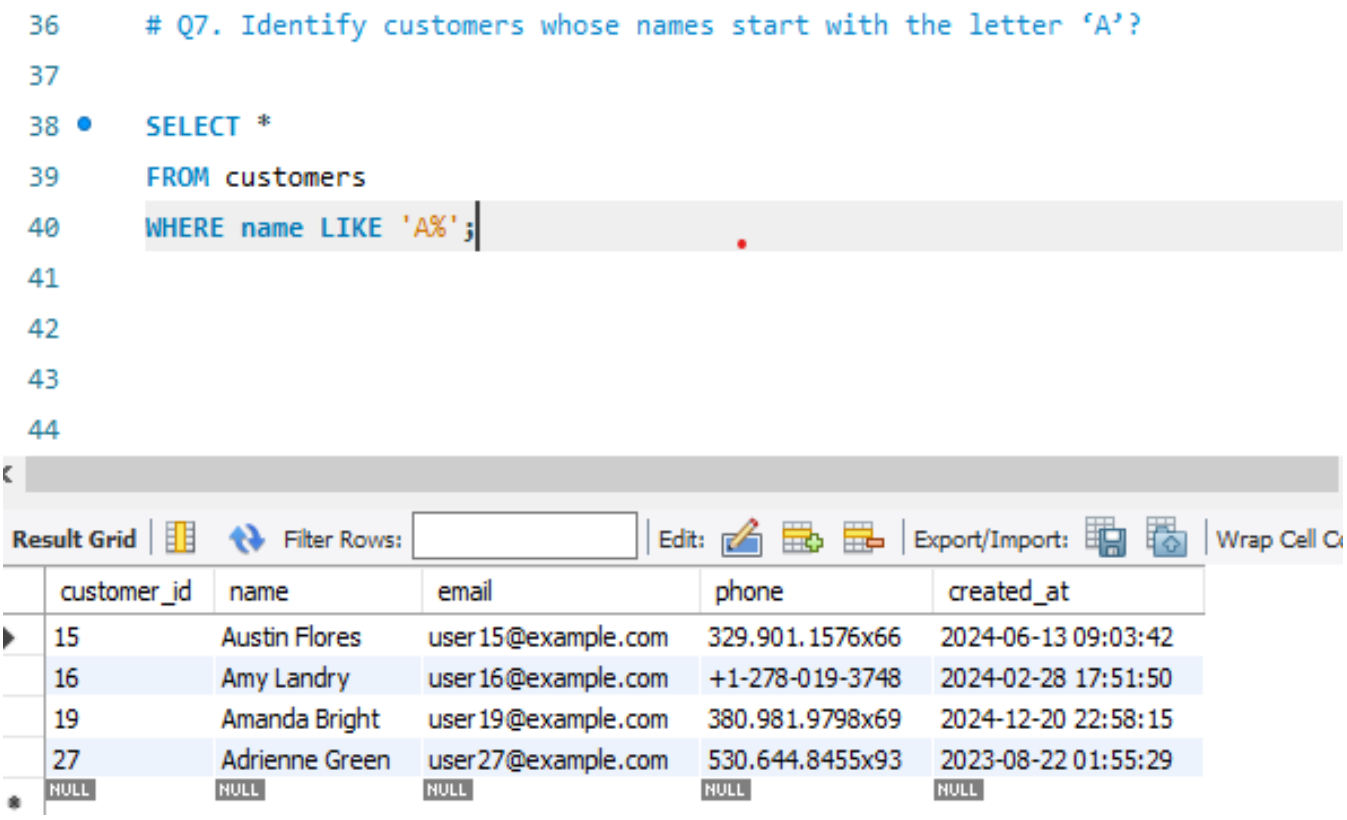
Used for alphabetical segmentation in outreach or app display

```
SELECT *

FROM customers

WHERE name LIKE 'A%';
```

OUTPUT(SCREENSHOT)



```
36    # Q7. Identify customers whose names start with the letter ‘A’?
37
38    SELECT *
39    FROM customers
40    WHERE name LIKE 'A%';
41
42
43
44
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell C

customer_id	name	email	phone	created_at
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
NULL	NULL	NULL	NULL	NULL

Q8. List electronics products priced under ₹3,000?

Used by merchandising or frontend teams to showcase budget electronic

```
SELECT *

FROM products

WHERE category = "electronics"

and price < 3000;
```

OUTPUT(SCREENSHOT)

```
46      # Q8. List electronics products priced under ₹3,000?
47
48 •    SELECT *
49      FROM products
50      WHERE category = "electronics"
51      and price < 3000;
52
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

product_id	name	category	price	stock_quantity	added_on
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
NULL	NULL	NULL	NULL	NULL	NULL

Q9. Display product names and prices in descending order of price?

This helps teams easily view and compare top-priced items.

```
SELECT name, price

FROM products

ORDER BY price desc;
```

OUTPUT(SCREENSHOT)

```
51      # Q9. Display product names and prices in descending order of price?
52
53 •    SELECT name, price
54      FROM products
55      ORDER BY price desc;
56
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08

Q10. Display product names and prices, sorted by price and then by name?

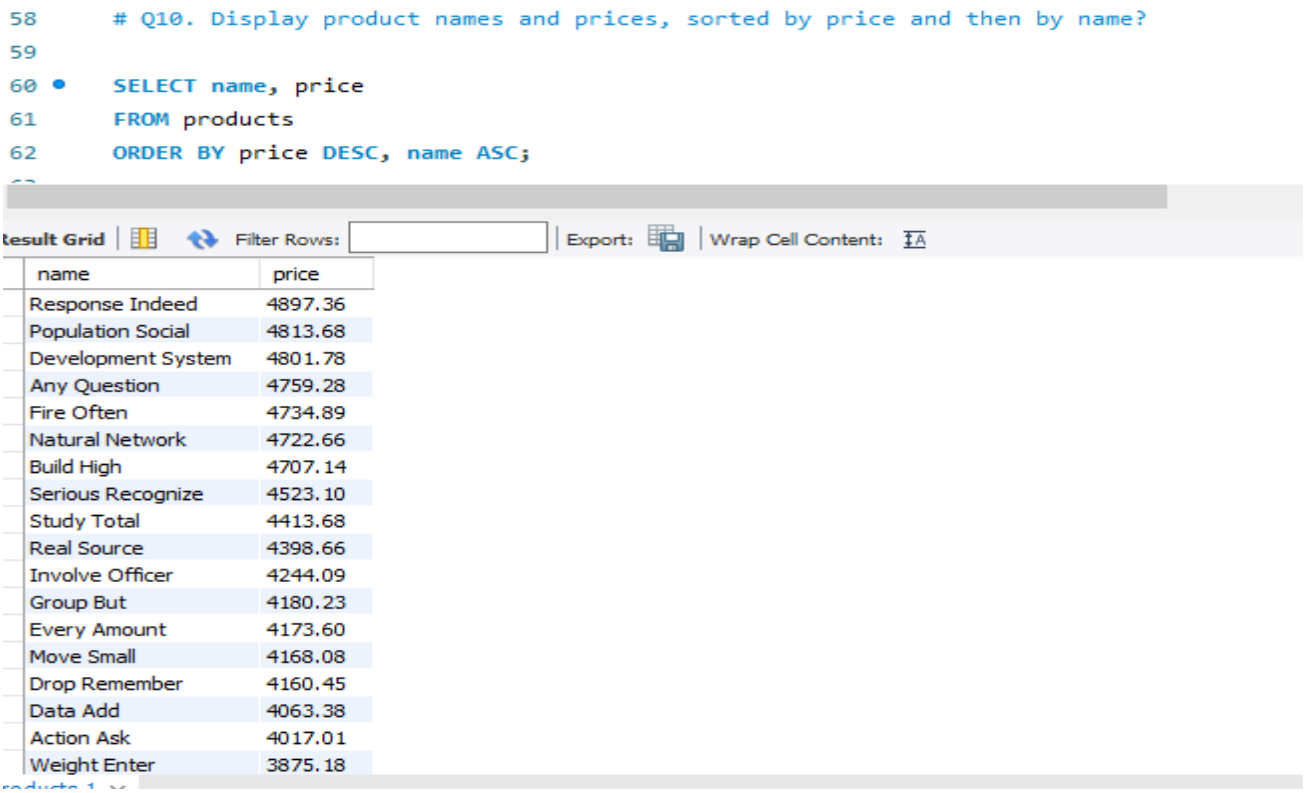
The merchandising or catalog team may want to list products from most expensive to cheapest. If multiple products have the same price, they should be sorted alphabetically for clarity on storefronts or printed catalogs.

```
SELECT name, price

FROM products

ORDER BY price DESC, name ASC;
```

OUTPUT(SCREENSHOT)



```
58      # Q10. Display product names and prices, sorted by price and then by name?
59
60 •    SELECT name, price
61      FROM products
62      ORDER BY price DESC, name ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01
Weight Enter	3875.18

LEVEL – 2 (FILTERING AND FORMATTING)

Q1.Retrieve orders where customer information is missing (possibly due to data migration ordeletion)?

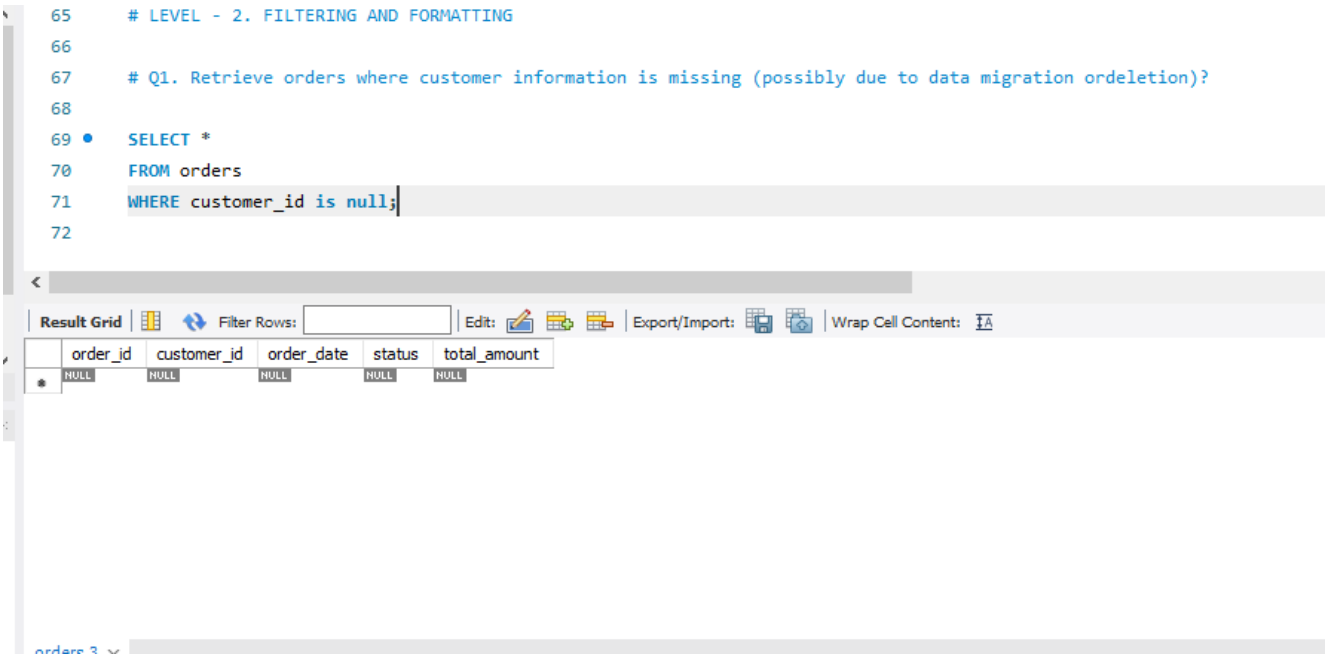
Used to identify orphaned orders or test data where customer_id is not linked

```
SELECT *

FROM orders

WHERE customer_id is null;
```

OUTPUT(SCREENSHOT)



Q2. Display customer names and emails using column aliases for frontend readability?

Useful for feeding into frontend displays or report headings that require user-friendly labels

```
SELECT
        name AS customer_name,
        email AS customer_emaiL
FROM customers;
```

OUTPUT(SCREENSHOT)

```
73    # Q2. Display customer names and emails using column aliases for frontend readability?
74
75 •  SELECT
76            name AS customer_name,
77            email AS customer_email
78    FROM customers;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	customer_email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com

Q3. Calculate total value per item ordered by multiplying quantity and item price?

This can help generate per-line item bill details or invoice breakdowns

```
SELECT *,
          quantity * item_price AS total_value
FROM order_items;
```

OUTPUT(SCREENSHOT)

```
80    # Q3. Calculate total value per item ordered by multiplying quantity and item price?
81
82    SELECT *,
83           quantity * item_price AS total_value
84    FROM order_items;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

order_item_id	order_id	product_id	quantity	item_price	total_value
1	1	19	2	4707.14	9414.28
2	2	6	3	177.40	532.20
3	3	9	3	985.19	2955.57
4	3	23	1	2208.99	2208.99
5	4	35	2	4734.89	9469.78
6	5	19	1	4707.14	4707.14
7	5	7	2	4897.36	9794.72
8	6	7	3	4897.36	14692.08
9	6	37	1	1429.45	1429.45
10	6	15	1	723.97	723.97
11	6	35	3	4734.89	14204.67

Q4. Combine customer name and phone number in a single column?

Used to show brief customer summaries or contact lists

```
SELECT CONCAT(name, ' - ' ,phone) AS customer_details

FROM customers;
```

OUTPUT(SCREENSHOT)

```
92      # Q4. Combine customer name and phone number in a single column?
93
94 •    SELECT CONCAT(name, ' - ' ,phone) AS customer_details
95      FROM customers;
96
97
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_details
Thomas Owens - 142-479-1945
Charles Grant - 9153947511
Kaitlin Richards - 2073473421
Christina Williams - 586-605-5061x06
David Allen - (751)456-8289x1
Mark Duke - (144)957-2811
Briana Wright - 223-833-9635
John Bryan - 045.568.0798x27
Jason Thompson - 1862659420
Shawn Hill - (268)113-3152x7
Walter Jenkins - 536-329-0817x71

Q5. Extract only the date part from order timestamps for date-wise reporting?

Helps group or filter orders by date without considering time.

```
SELECT order_id,
     DATE(order_date) AS order_date
FROM orders;
```

OUTPUT(SCREENSHOT)

```
91      # Q5. Extract only the date part from order timestamps for date-wise reporting?
92
93      SELECT order_id,
94          DATE(order_date) AS order_date
95      FROM orders;
96
97
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	order_date
1	2025-03-02
2	2024-10-09
3	2025-05-08
4	2024-09-19
5	2025-04-08
6	2024-10-25
7	2024-07-29
8	2024-07-30
9	2025-06-10
10	2025-02-16
11	2025-03-11

Q6. List products that do not have any stock left?

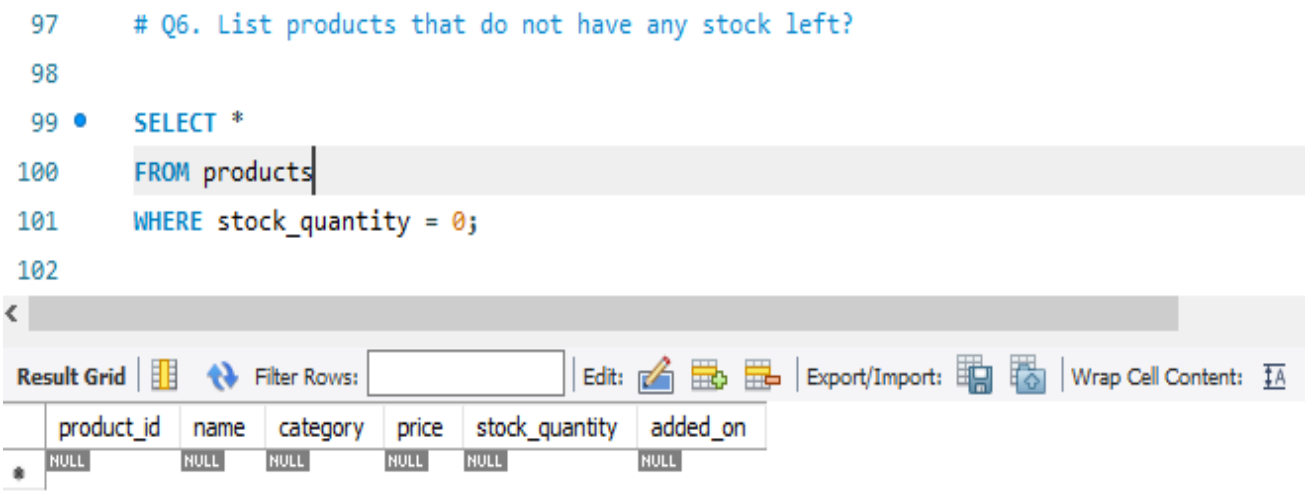
This helps the inventory team identify out-of-stock items.

```
SELECT *

FROM products

WHERE stock_quantity = 0;
```

OUTPUT(SCREENSHOT)



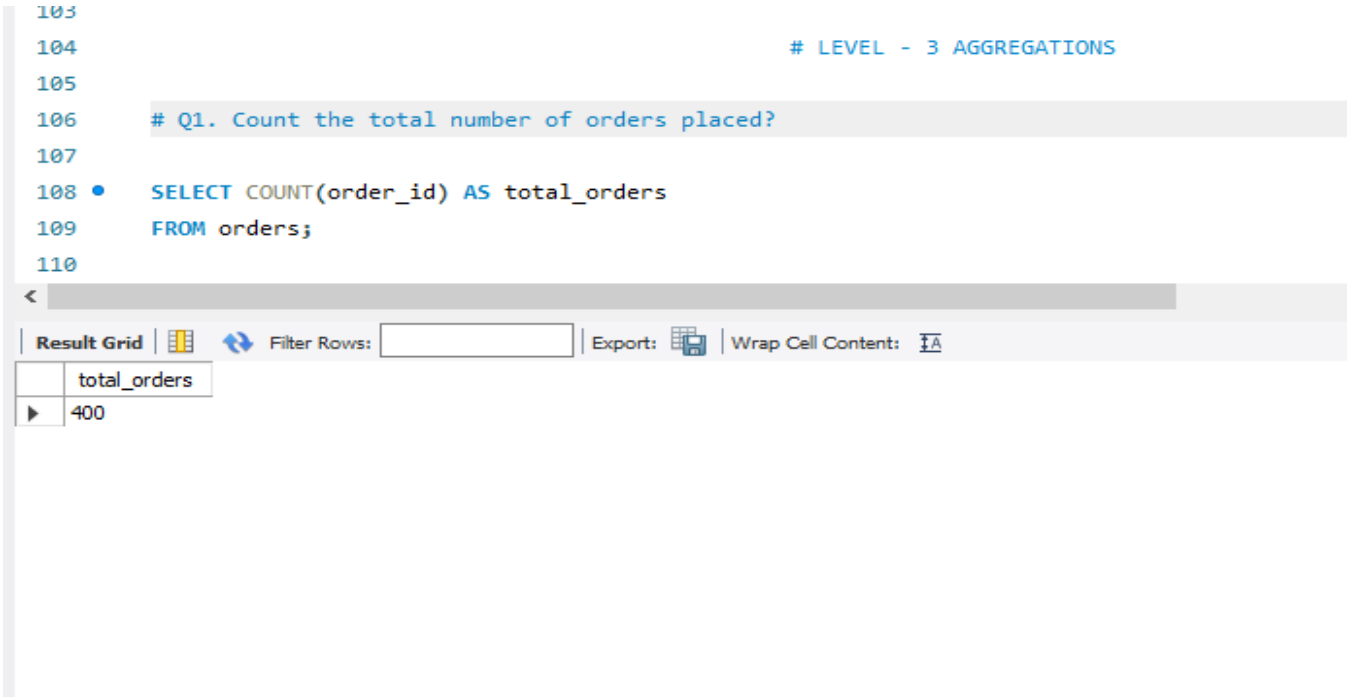
LEVEL – 3 (AGGREGATIONS)

Q1.Count the total number of orders placed?

Used by business managers to track order volume over time.

```
SELECT COUNT(order_id) AS total_orders
FROM orders;
```

OUTPUT(SCREENSHOT)



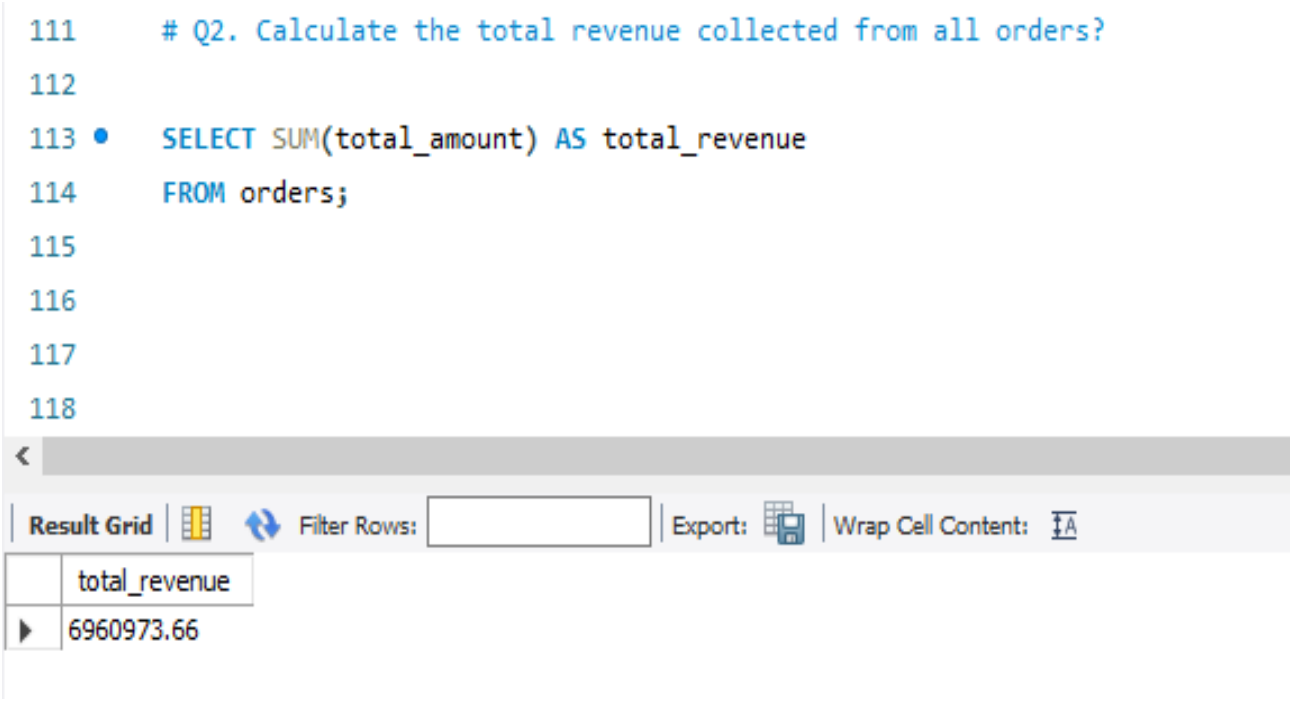
Q2. Calculate the total revenue collected from all orders?

This gives the overall sales value

```
SELECT SUM(total_amount) AS
total_revenue

FROM orders;
```

OUTPUT(SCREENSHOT)

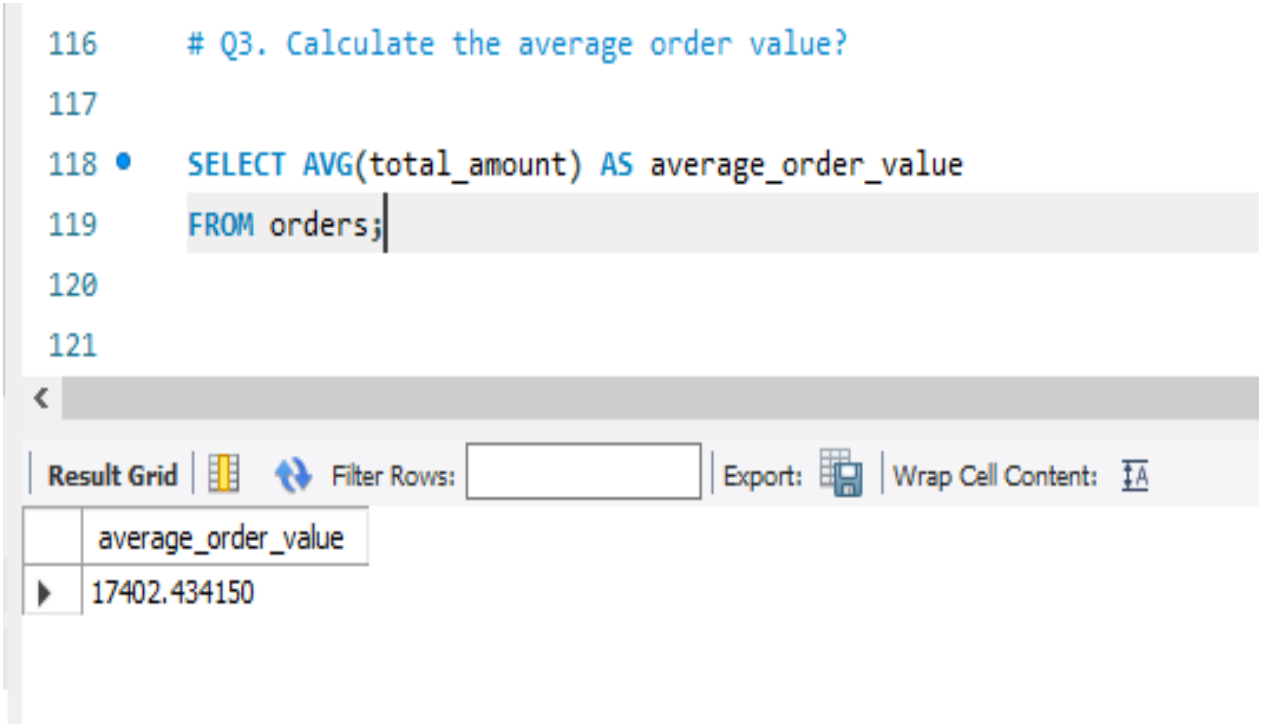


Q3. Calculate the average order value?

Used for understanding customer spending patterns

```
SELECT AVG(total_amount) AS average_order_value
FROM orders;
```

OUTPUT(SCREENSHOT)

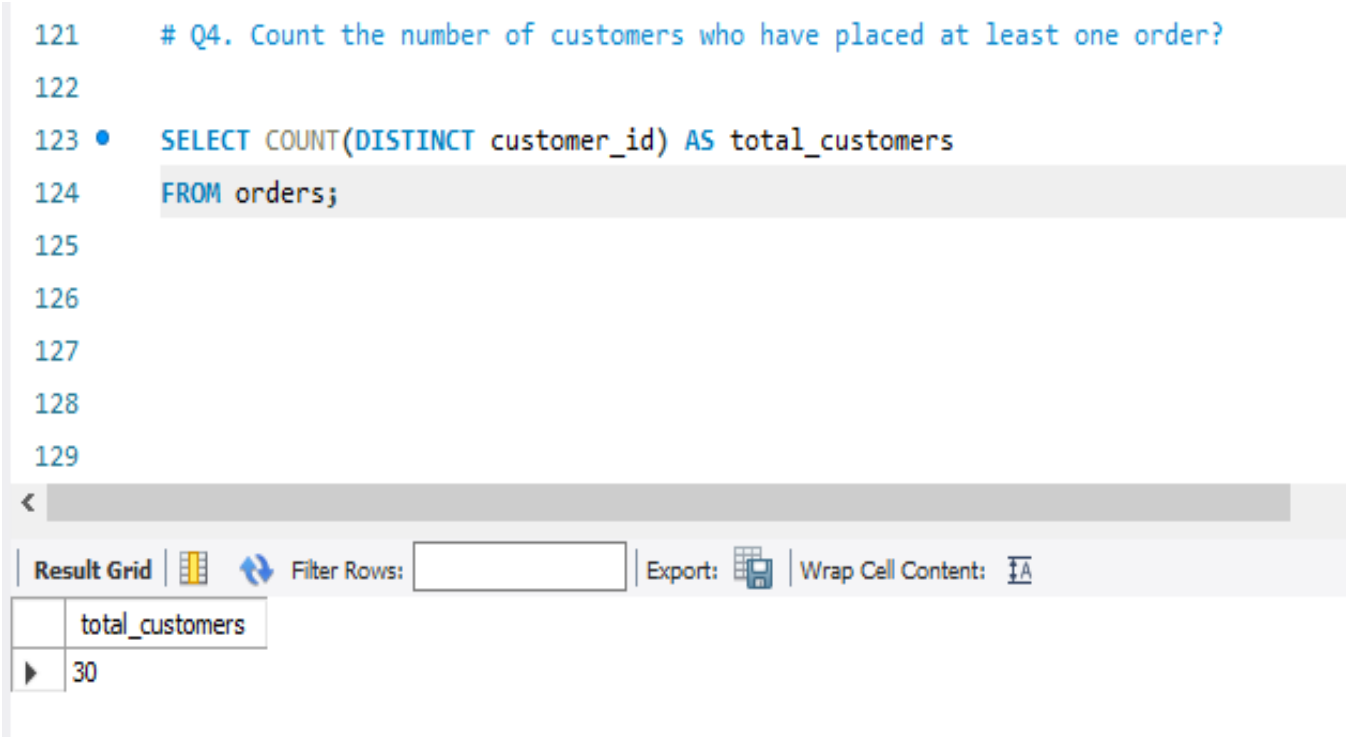


Q4. Count the number of customers who have placed at least one order?

This identifies active customers.

```
SELECT COUNT(DISTINCT customer_id) AS total_customers
FROM orders;
```

OUTPUT(SCREENSHOT)



Q5. Find the number of orders placed by each customer?

Helpful for identifying top or repeat customers

```
SELECT customer_id,
         COUNT(order_id) AS total_orders
FROM orders
GROUP BY customer_id;
```

OUTPUT(SCREENSHOT)

```
129     # Q5. Find the number of orders placed by each customer?
130
131 •   SELECT customer_id,
132             COUNT(order_id) AS total_orders
133     FROM orders
134     GROUP BY customer_id;
135
136
137
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_orders
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10
10	13
11	9

Q6. Find total sales amount made by each customer?

```
SELECT customer_id,
          SUM(total_amount) AS total_sales
FROM orders
GROUP BY customer_id;
```

OUTPUT(SCREENSHOT)

```
138     # Q6. Find total sales amount made by each customer?
139
140 •   SELECT customer_id,
141             SUM(total_amount) AS total_sales
142     FROM orders
143     GROUP BY customer_id;
144
145
146
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

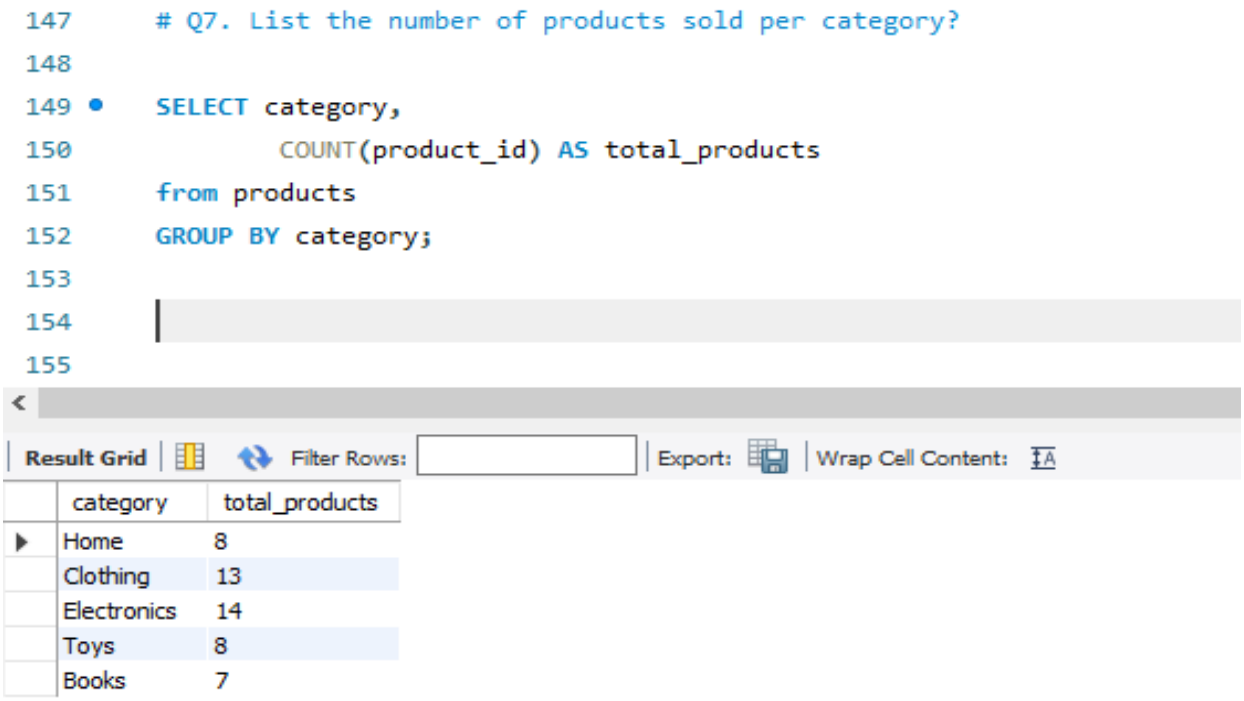
customer_id	total_sales
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75
11	173409.01

7. List the number of products sold per category?

This helps category managers assess performance by department.

```
SELECT category,
         COUNT(product_id) AS total_products
from products
GROUP BY category;
```

OUTPUT(SCREENSHOT)



```
147     # Q7. List the number of products sold per category?
148
149 •   SELECT category,
150             COUNT(product_id) AS total_products
151     from products
152     GROUP BY category;
153
154
155
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

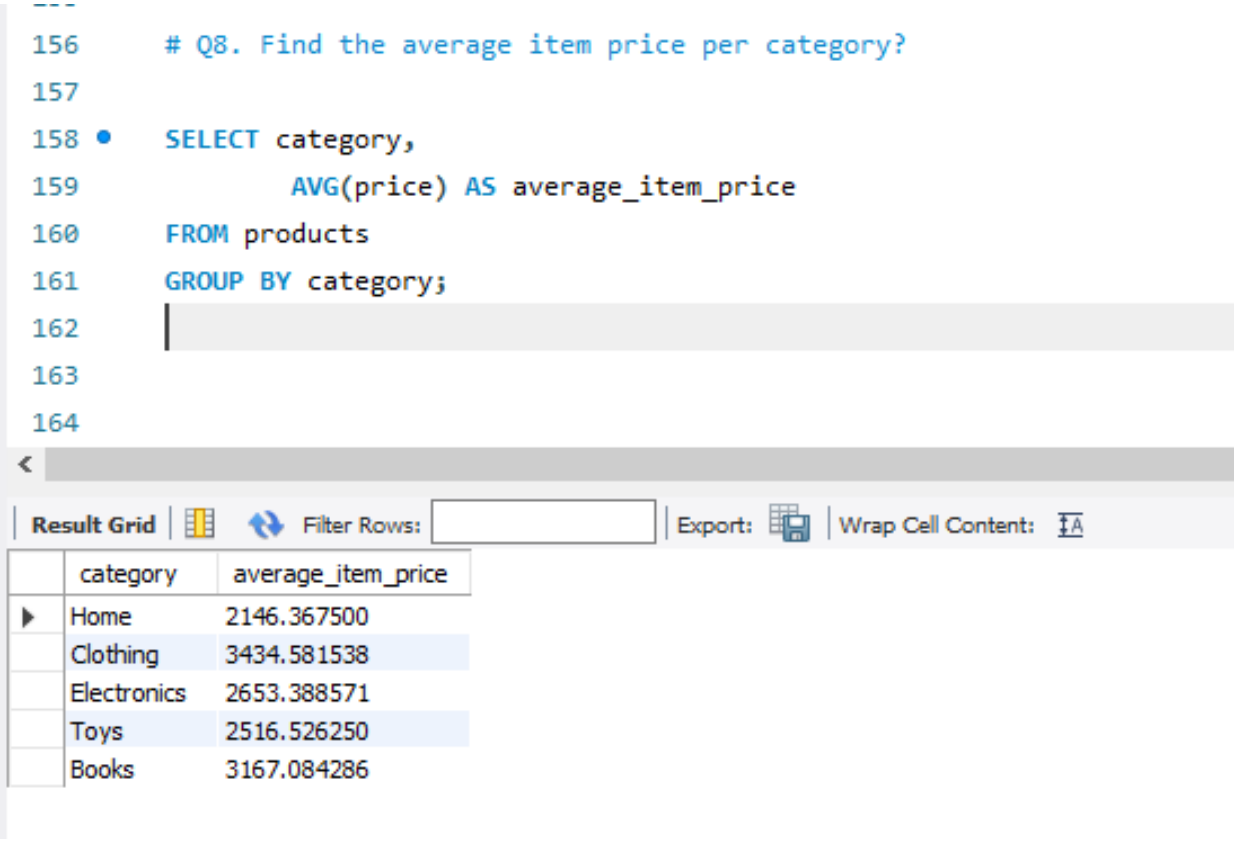
category	total_products
Home	8
Clothing	13
Electronics	14
Toys	8
Books	7

8. Find the average item price per category?

Useful to compare pricing across departments.

```
SELECT category,
          AVG(price) AS average_item_price
FROM products
GROUP BY category;
```

OUTPUT(SCREENSHOT)



```
156     # Q8. Find the average item price per category?
157
158 •   SELECT category,
159             AVG(price) AS average_item_price
160     FROM products
161     GROUP BY category;
162
163
164
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_item_price
Home	2146.367500
Clothing	3434.581538
Electronics	2653.388571
Toys	2516.526250
Books	3167.084286

9. Show number of orders placed per day?

Used to track daily business activity and demand trends

```
SELECT DATE(order_date) AS order_day,
         COUNT(order_id) AS total_orders
FROM orders
GROUP BY DATE(order_date);
```

OUTPUT(SCREENSHOT)

```
165     # Q9. Show number of orders placed per day?
166
167 •   SELECT DATE(order_date) AS order_day,
168             COUNT(order_id) AS total_orders
169     FROM orders
170     GROUP BY DATE(order_date)
171
172
173
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_day	total_orders
2025-03-02	3
2024-10-09	2
2025-05-08	2
2024-09-19	3
2025-04-08	3
2024-10-25	3
2024-07-29	2
2024-07-30	2
2025-06-10	2
2025-02-16	2
2025-03-11	3

Q10. List total payments received per payment method?

Helps the finance team understand preferred transaction modes

```
SELECT method,
          SUM(amount_paid) AS total_payments
FROM payments
GROUP BY method;
```

OUTPUT(SCREENSHOT)

```
172     # Q10. List total payments received per payment method?
173
174 •   SELECT method,
175             SUM(amount_paid) AS total_payments
176     FROM payments
177     GROUP BY method;
178
179
180
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

method	total_payments
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78
Debit Card	1930577.88

LEVEL – 4 MULTI-TABLE-QUERIES (JOINS)

Q1. Retrieve order details along with the customer name (INNER JOIN)?

Used for displaying which customer placed each order.

```
SELECT orders.*, customers.name

FROM customers

INNER JOIN orders

ON customers.customer_id = orders.customer_id;
```

OUTPUT(SCREENSHOT)

```
178
179                                          # LEVEL - 4 MUTLI-TABLE QUERIES (JOINS)
180
181     # Q1. Retrieve order details along with the customer name (INNER JOIN)?
182
183 •   SELECT orders.*, customers.name
184     FROM customers
185     INNER JOIN orders
186     ON customers.customer_id = orders.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount	name
14	1	2024-09-24 21:21:38	Shipped	15803.34	Thomas Owens
17	1	2024-08-19 21:17:57	Pending	11173.77	Thomas Owens
61	1	2024-12-26 23:21:58	Shipped	13053.00	Thomas Owens
76	1	2025-01-14 22:59:54	Pending	23506.81	Thomas Owens
92	1	2024-09-26 11:33:58	Shipped	834.75	Thomas Owens
109	1	2025-03-17 04:56:18	Shipped	12190.14	Thomas Owens
127	1	2025-06-10 18:01:32	Pending	20160.64	Thomas Owens
135	1	2025-03-11 22:58:23	Pending	8891.79	Thomas Owens
144	1	2024-09-19 09:18:22	Shipped	12093.54	Thomas Owens
221	1	2024-09-25 05:49:39	Pending	12437.61	Thomas Owens
278	1	2025-02-01 20:05:38	Delivered	32015.16	Thomas Owens
379	1	2025-03-22 15:57:58	Cancelled	21586.89	Thomas Owens
56	2	2024-06-23 15:59:57	Pending	6828.61	Charles Grant
169	2	2025-02-11 04:04:38	Cancelled	9426.30	Charles Grant

Q2. Get list of products that have been sold (INNER JOIN with order_items)?

Used to find which products were actually included in orders

```
SELECT DISTINCT products.*
FROM products
INNER JOIN order_items
ON products.product_id = order_items.product_id;
```

OUTPUR(SCREENSHOT)

```
188     # Q2. Get list of products that have been sold (INNER JOIN with order_items)?
189
190 •   SELECT DISTINCT products.*
191     FROM products
192     INNER JOIN order_items
193     ON products.product_id = order_items.product_id;
194
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
14	Listen Development	Home	296.17	132	2024-10-12 11:49:26

Q3. List all orders with their payment method (INNER JOIN)?

Used by finance or audit teams to see how each order was paid for.

```
SELECT orders.*, payments.method

FROM orders

INNER JOIN payments

ON orders.order_id = payments.order_id;
```

OUTPUT(SCREENSHOT)

```
195     # Q3. List all orders with their payment method (INNER JOIN)?
196
197 •   SELECT orders.*, payments.method
198     FROM orders
199     INNER JOIN payments
200     ON orders.order_id = payments.order_id;
201
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount	method
1	20	2025-03-02 07:20:11	Delivered	9414.28	Credit Card
2	18	2024-10-09 18:08:21	Shipped	532.20	Net Banking
3	15	2025-05-08 00:08:27	Cancelled	5164.56	Credit Card
4	11	2024-09-19 22:16:13	Delivered	9469.78	UPI
5	12	2025-04-08 18:02:06	Pending	14501.86	UPI
6	29	2024-10-25 07:33:59	Cancelled	31050.17	UPI
7	22	2024-07-29 11:58:47	Shipped	3043.67	UPI
8	19	2024-07-30 22:49:49	Cancelled	32714.06	Net Banking
9	6	2025-06-10 17:00:25	Pending	24219.20	Net Banking
10	28	2025-02-16 12:45:59	Delivered	24342.52	Debit Card
11	25	2025-03-11 14:22:46	Cancelled	16196.13	Credit Card
12	28	2025-01-15 13:22:53	Shipped	9890.72	Credit Card
13	25	2025-05-12 13:04:29	Cancelled	9627.36	UPI
14	1	2024-09-24 21:21:38	Shipped	15803.34	UPI

Q4. Get list of customers and their orders (LEFT JOIN)?

Used to find all customers and see who has or hasn't placed orders.

```
SELECT customers.*, orders.*

FROM customers

LEFT JOIN orders

ON customers.customer_id = orders.customer_id;
```

OUTPUT(SCREENSHOT)

```
202     # Q4. Get list of customers and their orders (LEFT JOIN)?
203
204 •   SELECT customers.*, orders.*
205     FROM customers
206     LEFT JOIN orders
207     ON customers.customer_id = orders.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	email	phone	created_at	order_id	customer_id	order_date	status	total_amount
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	1	2024-09-24 21:21:38	Shipped	15803.34
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	1	2024-08-19 21:17:57	Pending	11173.77
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	1	2024-12-26 23:21:58	Shipped	13053.00
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	1	2025-01-14 22:59:54	Pending	23506.81
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	1	2024-09-26 11:33:58	Shipped	834.75
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	1	2025-03-17 04:56:18	Shipped	12190.14
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	127	1	2025-06-10 18:01:32	Pending	20160.64
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	135	1	2025-03-11 22:58:23	Pending	8891.79
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	144	1	2024-09-19 09:18:22	Shipped	12093.54
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	221	1	2024-09-25 05:49:39	Pending	12437.61
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	278	1	2025-02-01 20:05:38	Delivered	32015.16
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	379	1	2025-03-22 15:57:58	Cancelled	21586.89
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	56	2	2024-06-23 15:59:57	Pending	6828.61
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	169	2	2025-02-11 04:04:38	Cancelled	9426.30
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24	209	2	2024-10-16 10:07:48	Shipped	7690.62

Q5. List all products along with order item quantity (LEFT JOIN)?

Useful for inventory teams to track what sold and what hasn't.

```
SELECT products.*, order_items.quantity
FROM products
LEFT JOIN order_items
ON products.product_id = order_items.product_id;
```

OUTPUT(SCREENSHOT)

```
210     # Q5. List all products along with order item quantity (LEFT JOIN)?
211
212 •   SELECT products.*, order_items.quantity
213     FROM products
214     LEFT JOIN order_items
215     ON products.product_id = order_items.product_id;
216
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

product_id	name	category	price	stock_quantity	added_on	quantity
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	2
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	2
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	2
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	3
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	3
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	2
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	2
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	2
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	3
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	1
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	1
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	3
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	1
1	Plant No	Home	639.43	152	2024-01-30 06:30:53	1
2	Populat...	Clothing	4813.68	84	2025-05-30 10:02:50	2
2	Populat...	Clothing	4813.68	84	2025-05-30 10:02:50	2
2	Populat...	Clothing	4813.68	84	2025-05-30 10:02:50	2
2	Populat...	Clothing	4813.68	84	2025-05-30 10:02:50	3

Q6. List all payments including those with no matching orders (RIGHT JOIN)?

Rare but used when ensuring all payments are mapped correctly.

```
SELECT orders.*, payments.*

FROM orders

RIGHT JOIN payments

ON orders.order_id = payments.order_id;
```

OUTPUT(SCREENSHOT)

```
218     # Q6. List all payments including those with no matching orders (RIGHT JOIN)?
219
220 •   SELECT orders.*, payments.*
221     FROM orders
222     RIGHT JOIN payments
223     ON orders.order_id = payments.order_id;
224
225
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount	payment_id	order_id	payment_date	amount_paid	method
1	20	2025-03-02 07:20:11	Delivered	9414.28	1	1	2025-03-02 08:16:11	9414.28	Credit Card
2	18	2024-10-09 18:08:21	Shipped	532.20	2	2	2024-10-09 19:00:21	532.20	Net Banking
3	15	2025-05-08 00:08:27	Cancelled	5164.56	3	3	2025-05-08 00:55:27	5164.56	Credit Card
4	11	2024-09-19 22:16:13	Delivered	9469.78	4	4	2024-09-19 22:28:13	9469.78	UPI
5	12	2025-04-08 18:02:06	Pending	14501.86	5	5	2025-04-08 18:26:06	14501.86	UPI
6	29	2024-10-25 07:33:59	Cancelled	31050.17	6	6	2024-10-25 08:16:59	31050.17	UPI
7	22	2024-07-29 11:58:47	Shipped	3043.67	7	7	2024-07-29 12:45:47	3043.67	UPI
8	19	2024-07-30 22:49:49	Cancelled	32714.06	8	8	2024-07-30 23:42:49	32714.06	Net Banking
9	6	2025-06-10 17:00:25	Pending	24219.20	9	9	2025-06-10 17:36:25	24219.20	Net Banking
10	28	2025-02-16 12:45:59	Delivered	24342.52	10	10	2025-02-16 13:43:59	24342.52	Debit Card
11	25	2025-03-11 14:22:46	Cancelled	16196.13	11	11	2025-03-11 15:05:46	16196.13	Credit Card
12	28	2025-01-15 13:22:53	Shipped	9890.72	12	12	2025-01-15 14:00:53	9890.72	Credit Card

Result 45

Q7. Combine data from three tables: customer, order, and payment?

Used for detailed transaction reports.

```
SELECT customers.*, orders.*, payments.*
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id
INNER JOIN payments
ON orders.order_id = payments.order_id;
```

OUTPUT(SCREENSHOT)

```
224
225    # Q7. Combine data from three tables: customer, order, and payment?
226
227 •  SELECT customers.*, orders.*, payments.*
228    FROM customers
229    INNER JOIN orders
230    ON customers.customer_id = orders.customer_id
231    INNER JOIN payments
232    ON orders.order_id = payments.order_id;
233
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	email	phone	created_at	order_id	customer_id	order_date	status	total_amount	payment_id	order_id	payment_date	amount_paid	method
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	14	1	2024-09-24 21:21:38	Shipped	15803.34	14	14	2024-09-24 22:11:38	15803.34	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	17	1	2024-08-19 21:17:57	Pending	11173.77	17	17	2024-08-19 21:49:57	11173.77	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	61	1	2024-12-26 23:21:58	Shipped	13053.00	61	61	2024-12-26 23:31:58	13053.00	Net Banking
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	76	1	2025-01-14 22:59:54	Pending	23506.81	76	76	2025-01-14 23:33:54	23506.81	Credit Card
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	92	1	2024-09-26 11:33:58	Shipped	834.75	92	92	2024-09-26 12:03:58	834.75	Net Banking
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	109	1	2025-03-17 04:56:18	Shipped	12190.14	109	109	2025-03-17 05:16:18	12190.14	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	127	1	2025-06-10 18:01:32	Pending	20160.64	127	127	2025-06-10 18:57:32	20160.64	Debit Card
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	135	1	2025-03-11 22:58:23	Pending	8891.79	135	135	2025-03-11 23:11:23	8891.79	Debit Card
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	144	1	2024-09-19 09:18:22	Shipped	12093.54	144	144	2024-09-19 10:10:22	12093.54	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	221	1	2024-09-25 05:49:39	Pending	12437.61	221	221	2024-09-25 06:25:39	12437.61	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	278	1	2025-02-01 20:05:38	Delivered	32015.16	278	278	2025-02-01 21:05:38	32015.16	UPI
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12	379	1	2025-03-22 15:57:58	Cancelled	21586.89	379	379	2025-03-22 16:48:58	21586.89	Credit Card

Result 46 x

LEVEL – 5 SUBQUERIES (INNER QUERIES)

Q1. List all products priced above the average product price?

Used by pricing analysts to identify premium-priced products.

```
SELECT product_id, name, price
FROM products
WHERE price > (
          SELECT AVG(price)
         FROM products
);
```

OUTPUT(SCREENSHOT)

```sql
# Q1. List all products priced above the average product price?

SELECT product_id, name, price
FROM products
WHERE price > (
        SELECT AVG(price)
        FROM products
);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Co

product_id	name	price
2	Population Social	4813.68
4	Any Question	4759.28
5	Natural Network	4722.66
7	Response Indeed	4897.36
8	Every Amount	4173.60
10	Development System	4801.78
12	Action Ask	4017.01
18	Some Them	3673.86
19	Build High	4707.14
20	Real Source	4398.66
21	Bed Including	3845.31
22	Move Small	4168.08
24	Assume Serve	3437.81
25	South Free	3543.58

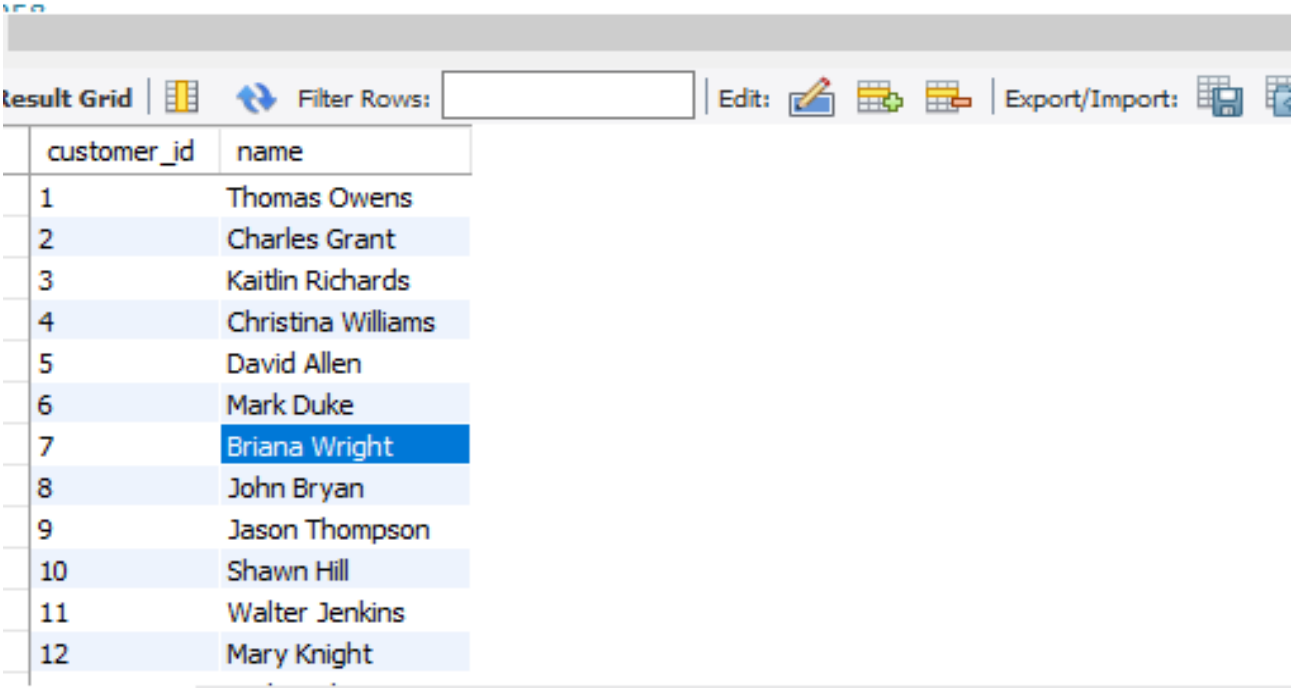
Q2. Find customers who have placed at least one order?

Used to identify active customers for loyalty campaigns.

```
SELECT customer_id, name
FROM customers c
WHERE EXISTS (
          SELECT 1
         FROM orders o
         WHERE o.customer_id =
c.customer_id
);
```

OUTPUT(SCREENSHOT)

```
249     # Q2. Find customers who have placed at least one order?
250
251     SELECT customer_id, name
252     FROM customers c
253     WHERE EXISTS (
254             SELECT 1
255             FROM orders o
256             WHERE o.customer_id = c.customer_id
257     );
```



Result Grid | Filter Rows: | Edit: | Export/Import:

customer_id	name
1	Thomas Owens
2	Charles Grant
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Bryan
9	Jason Thompson
10	Shawn Hill
11	Walter Jenkins
12	Mary Knight

Q3. Show orders whose total amount is above the average for that customer?

Used to detect unusually high purchases per customer.

```
SELECT order_id, customer_id, total_amount
FROM orders o
WHERE total_amount > (
              SELECT AVG(total_amount)
            FROM orders
            WHERE customer_id = o.customer_id
);
```

OUTPUT(SCREENSHOT)

```
263     # Q3. Show orders whose total amount is above the average for that customer?
264
265     SELECT order_id, customer_id, total_amount
266     FROM orders o
267     WHERE total_amount > (
268                 SELECT AVG(total_amount)
269                 FROM orders
270                 WHERE customer_id = o.customer_id
271     );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Conte

order_id	customer_id	total_amount
6	29	31050.17
8	19	32714.06
9	6	24219.20
10	28	24342.52
14	1	15803.34
16	5	22856.73
18	13	32001.24
21	22	25364.11
22	20	21281.44
24	6	22882.19
25	17	23749.66
26	24	35167.92

Q4. Display customers who haven’t placed any orders?

Used for re-engagement campaigns targeting inactive users.

```
SELECT customer_id, name
FROM customers c
WHERE NOT EXISTS (
          SELECT 1
         FROM orders o
         WHERE o.customer_id = c.customer_id
);
```

OUTPUT(SCREENSHOT)

```
276     # Q4. Display customers who haven’t placed any orders?
277
278 •   SELECT customer_id, name
279     FROM customers c
280     WHERE NOT EXISTS (
281             SELECT 1
282             FROM orders o
283             WHERE o.customer_id = c.customer_id
284     );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cor

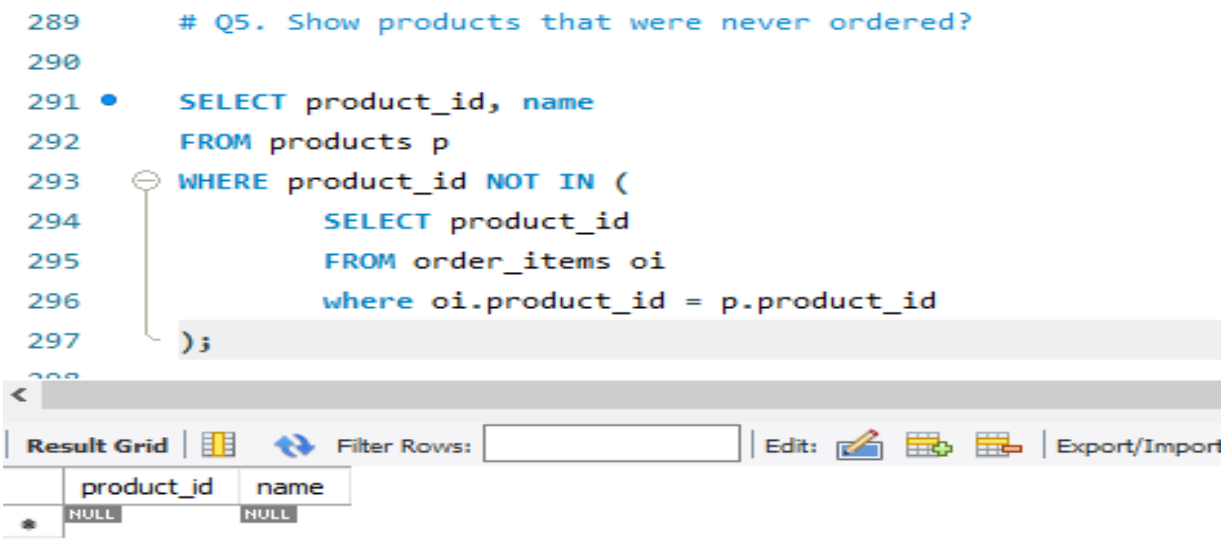
customer_id	name
NULL	NULL

Q5. Show products that were never ordered?

Helps with inventory clearance decisions or product deactivation

```
SELECT product_id, name
FROM products p
WHERE product_id NOT IN (
          SELECT product_id
         FROM order_items oi
         where oi.product_id = p.product_id
);
```

OUTPUT(SCREENSHOT)



Q6. Show highest value order per customer?

Used to identify the largest transaction made by each customer.

```
SELECT order_id, customer_id, total_amount
FROM orders o
WHERE total_amount = (
          SELECT MAX(total_amount)
         FROM orders
         WHERE customer_id = o.customer_id
);
```

OUTPUT(SCREENSHOT)

```sql
# Q6. Show highest value order per customer?

SELECT order_id, customer_id, total_amount
FROM orders o
WHERE total_amount = (
        SELECT MAX(total_amount)
        FROM orders
        WHERE customer_id = o.customer_id
);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

order_id	customer_id	total_amount
18	13	32001.24
33	8	36147.09
62	10	35723.17
66	27	26176.05
68	24	36159.92
80	12	38656.26
107	14	48042.06
121	16	37415.67
128	7	28589.04
133	11	37129.82
139	18	39857.19
174	19	35676.00
189	25	40872.29
196	21	24096.64

Q7. Highest Order Per Customer (Including Names)?

Used to identify the largest transaction made by each customer. Outputs name as well.

```
SELECT c.customer_id, c.name,
        o.order_id, o.total_amount
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.total_amount = (
          SELECT MAX(total_amount)
        FROM orders
        WHERE customer_id = c.customer_id
);
```

OUTPUT(SCREENSHOT)

```sql
# Q7. Highest Order Per Customer (Including Names)?

SELECT c.customer_id, c.name,
        o.order_id, o.total_amount
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.total_amount = (
        SELECT MAX(total_amount)
        FROM orders
        WHERE customer_id = c.customer_id
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	order_id	total_amount
1	Thomas Owens	278	32015.16
2	Charles Grant	315	42056.04
3	Kaitlin Richards	281	41679.11
4	Christina Williams	386	25747.34
5	David Allen	266	39921.78
6	Mark Duke	330	39003.19
7	Briana Wright	128	28589.04
8	John Bryan	33	36147.09
9	Jason Thompson	348	21414.44
10	Shawn Hill	62	35723.17
11	Walter Jenkins	133	37129.82
12	Mary Knight	80	38656.26

LEVEL – 6 SET OPERATIONS

Q1. List all customers who have either placed an order or written a product review?

Used to identify engaged customers from different activity areas.

```
SELECT c.customer_id,
       c.name
FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id
    FROM orders

    UNION

    SELECT customer_id
    FROM product_reviews
);
```

OUTPUT(SCREENSHOT)

```
328     # Q1. List all customers who have either placed an order or written a product review?
329
330     SELECT c.customer_id,
331            c.name
332     FROM customers c
333     WHERE c.customer_id IN (
334         SELECT customer_id
335         FROM orders
336
337         UNION
338
339         SELECT customer_id
340         FROM product_reviews
341     );
342
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

customer_id	name
1	Thomas Owens
2	Charles Grant
3	Kaitlin Richards
4	Christina Williams
5	David Allen
6	Mark Duke
7	Briana Wright
8	John Bryan
9	Jason Thompson
10	Shawn Hill
11	Walter Jenkins
12	Mary Knight

Q2. List all customers who have placed an order as well as reviewed a product [intersect notsupported]?

Used to identify highly engaged customers for rewards.

```
SELECT c.customer_id,
       c.name
FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id
    FROM orders
)
AND c.customer_id IN (
    SELECT customer_id
    FROM product_reviews
);
```

OUTPUT(SCREENSHOT)

```
344     # Q2. List all customers who have placed an order as well as reviewed a product [intersect notsupported]?
345
346     SELECT c.customer_id,
347            c.name
348     FROM customers c
349     WHERE c.customer_id IN (
350         SELECT customer_id
351         FROM orders
352     )
353     AND c.customer_id IN (
354         SELECT customer_id
355         FROM product_reviews
356     );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cor

customer_id	name
1	Thomas Owens
2	Charles Grant
4	Christina Williams
6	Mark Duke
7	Briana Wright
9	Jason Thompson
10	Shawn Hill
11	Walter Jenkins
13	Leslie Wilson
14	Deborah Arias
15	Austin Flores
17	Randy Mooney